
Zero shot Cross lingual Toxic Comment Detection: Calibrated Threshold Strategies for Multilingual Moderation

Group 5

BT5153 Applied Machine Learning for Business Analytics, National University of Singapore

Abstract

Online platforms increasingly require multilingual content moderation, but most labeled toxicity data is in English. We study zero shot cross lingual toxic comment detection on the Jigsaw Multilingual Toxic Comment Classification dataset, training all models on English data only and evaluating on Spanish, Italian, and Turkish. Four model families are compared in increasing complexity (TF IDF with Logistic Regression, BiLSTM, mBERT, XLM R), with class weighting (approximately 6.11 times for the positive class) used uniformly across all four. Beyond the model choice, we treat the decision threshold as a first class design problem and evaluate four strategies: a default 0.5 cutoff, a global recall constrained threshold tuned on the English dev set (Strategy B), an oracle per language threshold (C, leaky upper bound), and a leak free per language threshold tuned on a stratified validation split (D). We report two main findings. First, the gap in cross lingual ranking ability is large: XLM R reaches 0.8363 ROC AUC on the multilingual validation set while the English only baselines stay at 0.62 and 0.57. Second, Strategy B fails to meet its own recall target on every model, the result of a positive rate distribution shift from English dev (8.18 percent) to multilingual val (15.38 percent), whereas Strategy D recovers nearly all of the oracle gain (C minus D is at most 0.011 F1 across all four models). This identifies D as the recommended deployable operating point, with XLM R and D achieving F1 of 0.516 and FPR of 0.196 on the held out evaluation half.

1. Introduction

Automated toxicity classification has become a core component of online content moderation systems, but most published work and most labeled training data are concentrated in English. The same platforms host substantial volumes of non English content, and labels

for many of those languages are scarce or absent. This makes zero shot cross lingual generalization (training only on English and evaluating on unseen languages) a practically important question. We frame the task as binary classification: each comment receives a toxicity probability in $[0, 1]$, and a threshold converts this into a hard label.

Beyond the model itself, the threshold is the second design lever, and one that disproportionately affects deployment behavior. A moderation system tuned for high recall surfaces more harmful content but produces more false positives; one tuned for high precision misses real toxicity. The default 0.5 cutoff is rarely optimal under any distribution shift, and the right threshold may differ across languages because models tend to assign systematically different probability ranges per language even when their ranking quality is similar.

Building on the project proposal, this report addresses four research questions. (RQ1) Can a model trained only on English generalize to other languages, and how large is the performance gap across model families? (RQ2) Which threshold design (default, global, or per language) is most appropriate for a multilingual moderation setting, and how much of the gain from per language calibration survives when label leakage is removed? (RQ3) Which languages are hardest, and does this depend on the model? (RQ4) Are more complex models (transformers) actually worth the extra cost compared to simpler baselines?

Our contributions are: (i) a four model comparison under a uniform training and class weighting recipe; (ii) a four strategy threshold framework that separates the upper bound value of per language calibration (oracle) from its deployable, leak free value (proper); (iii) a key empirical finding that the canonical single threshold strategy tuned on English dev fails on multilingual validation due to a 2 times positive rate distribution shift; and (iv) a recommended operating point, XLM R combined with Strategy D, supported by a gap analysis showing that the val split noise penalty is at most 0.011 F1.

2. Dataset and Basic Facts

We use the Jigsaw Multilingual Toxic Comment Classification dataset (Jigsaw, 2020), which combines two English training sources and a multilingual validation and test set. The two training files (jigsaw toxic comment train.csv, a Wikipedia talk page corpus, and jigsaw unintended bias train.csv, the Civil Comments corpus

released for the unintended bias challenge (Jigsaw, 2019)) are both in English. The toxic comment train file contains binary labels directly. The unintended bias train file contains continuous toxicity scores in $[0, 1]$, which we binarize at 0.5 following the original challenge convention. After concatenating the two cleaned English sources and removing rows with missing or empty comment text and exact duplicates, we obtain a single English training pool, which we split 90 to 10 (stratified on the toxic label, random seed 42) into an English train portion and an English dev portion.

The official multilingual validation set is reserved for evaluation. It covers three languages, Spanish (es, n equals 2,500), Italian (it, n equals 2,500), and Turkish (tr, n equals 2,999), with a total of 7,999 rows. The official test set covers six languages and introduces three additional unseen languages (Russian, French, Portuguese), but ships without labels, so all numbers reported in this paper are computed on the labeled multilingual validation set.

Two facts shape the rest of the analysis and motivate our methodological choices. First, all subsets are class imbalanced. The positive (toxic) rate in the merged English training pool is 8.18 percent, and we apply a per class weighting of n total divided by (2 times n class) in every model, which yields a positive class weight of approximately 6.11 times. Second, and more consequentially, the positive rate on the multilingual validation set is 15.38 percent, roughly twice the training rate. This distribution shift means a classifier well calibrated to the English distribution will systematically under predict the positive class on validation, and, as we show in Section 4.2, it directly explains why a global threshold tuned on the English dev set fails to hold its recall target on multilingual data.

A first content level inspection of the English training data is also informative. After stop word removal, the most frequent words in toxic and non toxic comments overlap heavily on generic vocabulary (people, like, just, think). What distinguishes toxic comments is a small set of strongly indicative words, explicit slurs and insults, rather than a distinctive non toxic vocabulary. Toxic comments are also slightly shorter on average. This already suggests an asymmetry: toxic prediction relies heavily on the surface presence of offensive lexical items, which means a model that can only match exact English tokens will lose almost all of that signal once the test text is in Spanish, Italian, or Turkish.

3. Methods

3.1. Methodology

3.1.1. FOUR MODELS IN INCREASING COMPLEXITY

We compare four supervised classifiers in increasing order of expected cross lingual transfer ability: TF IDf with Logistic Regression, then BiLSTM, then mBERT, then XLM R. The Logistic Regression baseline uses scikit learn's TfidfVectorizer (1 to 2 grams, sublinear TF, accent stripping, `min_df` equals 3, `max_df` equals 0.95,

vocabulary cap 100k) and a balanced Logistic Regression with the liblinear solver. The BiLSTM uses a Keras tokenizer fit only on English (vocabulary cap 50k, OOV token reserved), pads to 128 tokens, and runs through an Embedding layer (dimension 128), a Bidirectional LSTM (64 units, dropout 0.2), a 64 unit ReLU dense layer, and a sigmoid output, trained with Adam (lr equals $1e-3$) for up to 5 epochs with early stopping (patience 2). mBERT is bert base multilingual cased, fine tuned for 2 epochs at lr equals $2e-5$ with weight decay 0.01, max length 128, train and eval batch size 16 and 32, and weighted cross entropy. XLM R is xlm roberta base, fine tuned under the same recipe but with train and eval batch size halved to 8 and 16 to fit GPU memory.

3.1.2. CLASS IMBALANCE HANDLING

All four models use class weighting based on the merged English training distribution (positive rate 8.18 percent). The weight for class c is computed as n total divided by (2 times n_c), giving a positive class weight of approximately 6.11 times and a negative class weight of approximately 0.54 times. For Logistic Regression we pass `class_weight` equals 'balanced'. For the BiLSTM we pass `class_weight` to `model.fit`. For mBERT and XLM R we override `compute_loss` inside a custom Trainer subclass to apply `CrossEntropyLoss` with weight equals $[0.54, 6.11]$. Using a uniform weighting recipe across all four models ensures that performance differences come from model capacity and pretraining, not from incompatible loss functions.

3.1.3. THRESHOLD OPTIMIZATION, FOUR STRATEGIES

Given the per comment probabilities, the choice of threshold is its own design problem. We evaluate four strategies, all using the same per model probability outputs and a unified tuning criterion: among thresholds achieving recall greater than or equal to 0.7, pick the one that maximizes F1.

- **A, Default 0.5:** the naive cutoff applied directly to multilingual val. Reference baseline.
- **B, Global recall:** one threshold per model, tuned on the English dev set under the recall greater than or equal to 0.7 and max F1 criterion. Leak free and deployable; one global threshold ships without language detection.
- **C, Per language oracle:** one threshold per (model, language), tuned and evaluated on the same multilingual val labels. Leaky by design; serves as the intentional upper bound.
- **D, Per language proper:** the multilingual val set is split 50 to 50, stratified on (language by toxic). Per language thresholds are tuned on the `val_tune` half (about 4,000 rows) and evaluated on the `val_eval` half (about 4,000 rows). Leak free and deployable, but uses half as much per language data as C, so it is noisier.

3.1.4. WHY D OVER B: DISTRIBUTION SHIFT BREAKS B

Strategy B is the natural deployable design: tune one

threshold on labeled English dev data, ship it. We find that B fails on every model in our comparison. Table 3 in Section 4.2 shows that under B, no model achieves the 0.7 recall target on multilingual val: actual recalls range from 0.043 (LR) to 0.483 (XLM R). The mechanism is direct: the English dev positive rate is 8.18 percent while the multilingual val positive rate is 15.38 percent, a 1.88 times shift. A threshold that produces a given recall on the lower positive distribution will produce a different (and on this dataset, much lower) recall once the positive distribution shifts up, because the additional positive mass sits at probabilities the threshold has not been calibrated against. This is the key methodological finding of the paper: the single threshold deployment story fails not because of model weakness, but because of distribution shift between the threshold tuning data and the deployment data.

3.1.5. GAP ANALYSIS VALIDATES D

Strategy D is the leak free per language counterpart. Table 4 shows the gap analysis for F1: D minus B (the realistic, deployable lift over B), C minus B (the oracle upper bound), and C minus D (the noise introduced by the val split). The C minus D gap is at most 0.011 F1 across all four models (LR 0.011, BiLSTM 0.002, mBERT 0.006, XLM R 0.009), which means the val split noise penalty is negligible. D therefore captures essentially all of C's gain without using validation labels twice. This is the empirical justification for selecting Strategy D as our recommended operating point.

3.2. Data preprocessing

Our preprocessing pipeline is intentionally light. Aggressive cleaning of toxicity data is risky because punctuation patterns (exclamation marks, ALL CAPS, repeated letters) and short function words often carry stylistic toxicity signals that a downstream model would otherwise rely on. We therefore restrict the pipeline to operations that fix structural inconsistencies between the four source files (column naming, label format, missing rows, duplicates) without altering surface form beyond whitespace normalization. The full pipeline applies in a fixed order across all three labeled datasets (toxic_train, unintended_bias_train, validation) and the unlabeled test file:

Step 1, Column standardization. The two English training files use inconsistent text column names: jigsaw_toxic_comment_train uses 'comment_text', while jigsaw_unintended_bias_train uses 'content'. We rename 'content' to 'comment_text' so that all four files share a single text column name.

Step 2, Column projection. We retain only the columns required downstream. The two training files keep (id, comment_text, toxic). The validation file additionally keeps the 'lang' column to support per language evaluation. The test file keeps (id, comment_text, lang) since it has no labels. Other auxiliary columns from the unintended bias file

(identity attributes, sub category scores) are dropped.

Step 3, Label binarization. The unintended bias training file ships continuous toxicity scores in [0, 1] rather than binary labels. We binarize these at threshold 0.5 following the original Civil Comments challenge convention. The other two labeled files (toxic_train, validation) already contain binary {0, 1} labels and require no conversion.

Step 4, Light text cleaning. We apply a single basic_text_clean function to the comment_text column of every dataset. The function casts non null inputs to str, strips leading and trailing whitespace, and collapses internal repeated whitespace into single spaces using the regular expression \s+ to space. We deliberately do not lowercase, do not remove punctuation, do not stem or lemmatize, and do not remove stopwords.

Step 5, Missing and empty text removal. We drop rows whose comment_text is null. We then cast comment_text to str and drop rows whose stripped value is the empty string. This catches both genuinely missing text and rows that consisted solely of whitespace before Step 4.

Step 6, Deduplication. We drop exact duplicate rows on dataset specific key columns: (comment_text, toxic) for English training, (comment_text, toxic, lang) for validation, and (comment_text, lang) for test. The validation key includes 'lang' because the same English token sequence (rare but possible) could legitimately appear under two different language labels and should not be merged.

Step 7, Type enforcement. We cast the toxic column to int across all labeled datasets to guard against silent float labels which would interact poorly with class weighted loss.

Step 8, English training merge. We concatenate the cleaned toxic_train and unintended_bias_train into a single English training pool, train_english_full. This is the only training data fed to the four classifiers.

Step 9, Train and dev split. We split train_english_full into a 90 percent train portion and a 10 percent dev portion using sklearn train_test_split with random_state equals 42 and stratification on the toxic label. The dev portion serves two roles: early stopping for the neural models, and the only label source consumed by Strategy B in Section 3.1.3.

Step 10, Sanity assertions and export. Before saving, we assert that comment_text exists in every dataset, that toxic labels are a subset of {0, 1}, and that 'lang' exists in validation and test. The five resulting tables (toxic_train_clean, unintended_bias_train_clean, train_english_merged_clean, validation_clean, test_clean) are written to a cleaned_data directory and reused by every downstream module without re running the cleaning pipeline.

We treat steps 1 to 7 as cleaning and steps 8 to 10 as dataset construction. The separation matters

because cleaning is dataset agnostic (the same pipeline applies to all four files) while dataset construction encodes our specific zero shot cross lingual setup, where the multilingual validation set is held out untouched and never used for training.

3.3. Evaluation metrics

We report two probability level metrics (ROC AUC and PR AUC), which depend only on the predicted ranking and are independent of any threshold, and four threshold level metrics (precision, recall, F1, and FPR). Under heavy class imbalance, ROC AUC can be optimistic; we therefore report PR AUC alongside it as a more conservative measure. All metrics are reported overall and broken down by language.

4. Results and Insights

4.1. Cross lingual ranking ability differs sharply across model families

Table 1 reports overall ROC AUC and PR AUC on the multilingual validation set. The ordering by overall AUC is XLM R (0.8363) greater than mBERT (0.7259) greater than LR (0.6223) greater than BiLSTM (0.5679). PR AUC tells the same story but at uniformly lower absolute values, as expected under 15.38 percent class imbalance: XLM R 0.4341, mBERT 0.3242, LR 0.2459, BiLSTM 0.2375. The gap between the multilingual transformers and the English only models is large and consistent.

Table 1. Overall ROC AUC and PR AUC on the multilingual validation set (n equals 7,999, positive rate 15.38 percent).

MODEL	ROC AUC	PR AUC
LR	0.6223	0.2459
BiLSTM	0.5679	0.2375
mBERT	0.7259	0.3242
XLM-R	0.8363	0.4341

Per language ROC AUC (Table 2) shows the same pattern at finer resolution. LR and BiLSTM are essentially monolingual, with their per language AUCs clustering in the 0.52 to 0.66 range, barely above chance. mBERT reaches 0.68 to 0.76, and XLM R reaches 0.78 to 0.93, with the strongest result on Turkish (0.9259). This directly answers RQ1 and RQ4: a model trained only on English can generalize to other languages, but only if its pretraining has already exposed it to those languages, and the gap from baselines to multilingual transformers is large enough to justify the extra training cost.

Table 2. Per language ROC AUC on the multilingual validation set.

MODEL	es	it	tr
LR	0.5865	0.6151	0.6608
BiLSTM	0.6417	0.5246	0.5578

MODEL	es	it	tr
mBERT	0.7587	0.6823	0.7272
XLM-R	0.8135	0.7753	0.9259

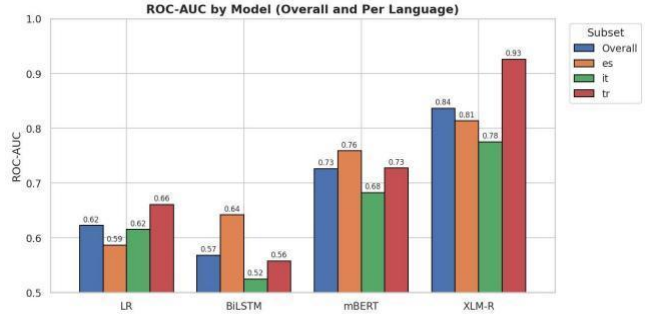


Figure 1. ROC AUC by model on the multilingual validation set, overall and per language.

4.2. Strategy B fails its own recall target

Table 3 reports the threshold level metrics for Strategy B (global threshold tuned on English dev for recall greater than or equal to 0.7) on the full multilingual validation set. The most striking column is recall: no model holds the 0.7 target. LR achieves recall 0.043, BiLSTM 0.077, mBERT 0.283, and XLM R 0.483, all far below 0.7. The selected thresholds are highly model dependent (LR 0.726, BiLSTM 0.839, mBERT 0.399, XLM R 0.037), reflecting the very different probability scales each model produces. Even at these tuned thresholds, the recall collapse on multilingual val is severe.

Table 3. Strategy B (global recall at 0.7) on the full multilingual val. None of the four models reaches the 0.7 recall target.

MODEL	Thr.	Prec.	Recall	F1	FPR
LR	0.7262	0.5096	0.0431	0.0795	0.0075
BiLSTM	0.8387	0.3942	0.0772	0.1292	0.0216
mBERT	0.3987	0.3941	0.2829	0.3294	0.0790
XLM-R	0.0373	0.4608	0.4829	0.4716	0.1027

The mechanism is the distribution shift identified in Section 3.1.4: the English dev set has 8.18 percent positives, the multilingual val set has 15.38 percent. A threshold tuned on the lower positive distribution does not retain its recall behavior once the positive mass shifts. This is not a model quality problem (XLM R has the best probabilistic ranking of the four, AUC 0.8363, and still loses 0.22 points of recall against its tuning target). It is a calibration versus shift problem, and the global single threshold design cannot solve it.

4.3. Per language calibration recovers the gap, and D matches C

Table 4 reports the F1 gap analysis across the four models. Two patterns are clear. First, both per language strategies (C and D) deliver large lifts over B; the realistic deployable lift D minus B ranges from 0.044 (XLM R) up to 0.217 (LR). Second, the gap between the oracle C and the leak free D is uniformly small: at most 0.011 F1 across all four models. This means the noise introduced

by the 50 to 50 val split is negligible, and a deployable per language tuning procedure recovers essentially all of the oracle gain.

Table 4. F1 gap analysis. D minus B is the deployable lift over Strategy B; C minus B is the oracle upper bound; C minus D is the val split noise. All C minus D values are at most 0.011, validating D as a leak free operating point.

MODEL	B(F1)	C(F1)	D(F1)	D minus B	C minus B	C minus D
LR	0.0795	0.3076	0.2967	+0.2173	+0.2282	0.0109
BiLSTM	0.1292	0.2774	0.2758	+0.1467	+0.1482	0.0015
mBERT	0.3294	0.3889	0.3831	+0.0538	+0.0595	0.0058
XLM-R	0.4716	0.5249	0.5157	+0.0441	+0.0533	0.0092

Table 5 reports the threshold level metrics for Strategy D, evaluated on the held out val_eval half (n equals 4,000, positive rate 15.375 percent). XLM R combined with D is the strongest deployable configuration: F1 of 0.5157, precision 0.4011, recall 0.7220, FPR 0.1959. mBERT combined with D reaches F1 of 0.3831 at FPR 0.3793. The two English only baselines reach the recall target by accepting an extreme false positive rate (LR FPR 0.5746; BiLSTM FPR 0.8579), which is not deployable in a real moderation pipeline because they pay for recall by labeling most non toxic content as toxic.

Table 5. Strategy D (per language proper) on val_eval (n equals 4,000, positive rate 15.375 percent). XLM R with D is the recommended deployable configuration.

MODEL	Prec.	Recall	F1	FPR
LR	0.1865	0.7252	0.2967	0.5746
BiLSTM	0.1624	0.9154	0.2758	0.8579
mBERT	0.2595	0.7317	0.3831	0.3793
XLM-R	0.4011	0.7220	0.5157	0.1959

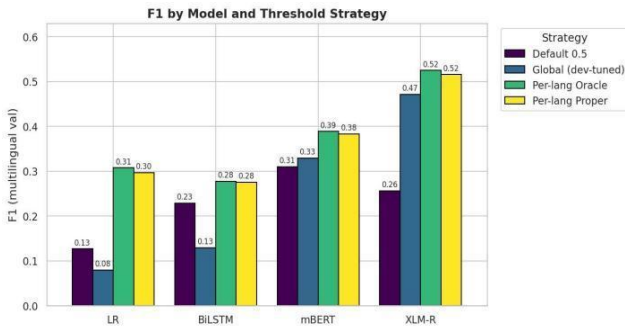


Figure 2. F1 on the multilingual validation set across four threshold strategies.

4.4. The recall and FPR trade off is dominated by the model

Figure 3 plots each (model, strategy) pair on the recall and FPR plane. Two patterns emerge. First, points belonging to the same model lie roughly on a single ROC curve through the origin: threshold tuning moves you along that curve, not above it. Second, the slope of that line is determined by the model. XLM R sits closest to the

upper left across strategies, achieving recall above 0.72 at FPR around 0.20 once per language thresholds are used. mBERT sits on a less favorable line. LR and BiLSTM lie close to the diagonal under aggressive thresholding: their high recall under Strategies C and D is bought at near equal cost in false alarms, which is why the bare F1 numbers in Table 4 underestimate how unusable they are in deployment. This view answers RQ4 directly: more complex models pay off because their recall and FPR frontier is genuinely better, not because the threshold is doing extra work.

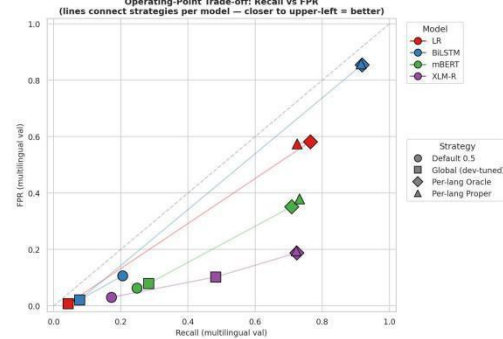


Figure 3. Recall versus FPR on the multilingual val set. Threshold strategies move points along each model's ROC curve; the curve itself is set by the model.

4.5. Per language F1 reveals where each model fails

Figure 4 shows per language F1 under Strategy D. XLM R is consistently strong, with F1 above 0.48 in every language and the highest score on Turkish (0.628). mBERT is roughly 0.10 lower per language and weakest on Turkish (0.308). LR and BiLSTM hover in the 0.19 to 0.35 range; BiLSTM in particular collapses on Turkish (F1 of 0.191), which is consistent with non Latin script Turkish tokens being largely out of vocabulary in its English trained tokenizer. For mBERT and XLM R, Italian is the weakest language (XLM R AUC equals 0.7753 on Italian versus 0.9259 on Turkish); one possible explanation is uneven toxic domain coverage in the Italian pretraining slice rather than a more general weakness on Romance languages, though confirming this would require a dedicated probing experiment beyond the scope of this report. Notably, the right model can flip which language is hardest: for XLM R, Turkish (a typologically distant agglutinative language) is the easiest setting, not the hardest.

Per-Language F1 under Per-Language Proper Tuning (Strategy D)

Model	es	it	tr
LR	0.289	0.353	0.251
BiLSTM	0.334	0.331	0.191
mBERT	0.446	0.400	0.308
XLM-R	0.486	0.484	0.628

Figure 4. Per language F1 under Strategy D (per language proper).

4.6 Test Set Results: XLM-R only

Because XLM-R achieved the strongest validation performance in both ranking metrics and threshold-level metrics, we additionally evaluated only XLM-R on the original test set. This test analysis is intended as a final robustness check for the selected model and thresholding design, rather than a repeated comparison across all four models. This follows the earlier validation results, where XLM-R had the highest overall ROC AUC, the strongest Strategy D F1, and the most favorable recall-FPR trade-off.

The original test set contains 63,812 comments across six languages. Three languages, Spanish, Italian, and Turkish, were already included in the multilingual validation set. For these languages, we applied the Strategy D per-language thresholds learned from the validation split. The other three languages, French, Portuguese, and Russian, were not available during validation threshold tuning. For these unseen languages, we applied the Strategy B global threshold as a fallback. In total, 30,932 test rows used per-language thresholds, while 32,880 test rows used the global fallback threshold. This creates a realistic deployment setting where calibrated languages use language-specific thresholds, while newly added languages temporarily rely on the global threshold.

Table 6 reports the overall test result. XLM-R achieves 0.8264 ROC AUC and 0.5524 PR AUC on the original test set, showing that its ranking ability remains strong beyond the validation set. At the selected operating thresholds, the model achieves an F1 score of 0.5447 with a false positive rate of 0.1407. The model predicts 14,949 out of 63,812 comments as toxic, giving a positive prediction rate of 23.43%. Precision is 0.5349 and recall is 0.5549, indicating a relatively balanced but still imperfect operating point.

Table 6. XLM-R test result summary under Strategy D with global fallback.

Metric	Value
Model	XLM-R
Setting	Strategy D + global fallback
Test rows	63,812
ROC AUC	0.8264
PR AUC	0.5524
F1	0.5447
FPR	0.1407

These results are consistent with the main validation findings. First, XLM-R remains the strongest practical model for cross-lingual toxicity detection, supporting the conclusion from RQ1 and RQ4 that multilingual pretraining is necessary for this task. Second, the

mixed thresholding result also supports the thresholding conclusion from RQ2. Per-language calibration is useful when labeled calibration data is available, but unseen languages still require a fallback rule. The fallback-language results suggest that the Strategy B global threshold should be treated as a temporary deployment solution rather than the final operating point. In practice, once enough labeled examples are collected for a new language, the system should switch from the global fallback threshold to a Strategy D per-language threshold.

4.7. Where the LR baseline still helps: interpretability

Even though the LR baseline is not competitive on multilingual F1, its top positive features remain a useful sanity check. The most positive English coefficients are exactly the kind of explicit insults and slurs that drive toxic predictions on English data; top negative coefficients are mostly neutral discourse markers. This confirms our earlier hypothesis: classification on this dataset is largely driven by the surface presence of offensive lexical items, which makes the dramatic drop from English only baselines to multilingual val entirely explicable, since those exact tokens simply do not appear in Spanish, Italian, or Turkish text.

4.8. Practical takeaways for deployment

The four findings above translate into a small set of concrete recommendations for teams building multilingual moderation systems. We summarize them here so that the chain from evidence to action is explicit, and so that readers who are not training their own models can still extract value from the threshold side of the analysis.

First, the model decision is binary in practice: either use a multilingually pretrained encoder (XLM R first, mBERT as a cheaper fallback) or accept that non English content will not be moderated. The English only baselines are not borderline; their per language F1 stays in the 0.19 to 0.35 range under every strategy we tested, and even their best operating points trade recall for FPR at near 1 to 1 ratios. There is no threshold tweak that turns LR or BiLSTM into a deployable multilingual classifier on this data. The cost of using XLM R over mBERT is modest (a 2 times larger encoder, 8 versus 16 batch size for the same hardware), and the F1 gain is roughly 0.13 absolute, which is the difference between a usable and an unusable system on Italian.

Second, the threshold decision should be made before the model is shipped, not after. Our results show that the same XLM R model produces F1 of 0.26 under Strategy A, F1 of 0.47 under B, and F1 of 0.52 under D on the same validation data: the only thing that changed is the threshold. Teams that ship at 0.5 by default are leaving roughly half of the model's effective performance on the table. If labeled per language data is available, Strategy D is the right choice and recovers essentially all of the oracle gain. If only English labeled data is available, Strategy B is the deployable fallback, but with the explicit

understanding that the resulting recall on non English content will be substantially lower than the recall floor used for tuning.

Third, evaluation should be per language, not pooled. A pooled F1 number on the multilingual val set hides the fact that XLM R's worst language (Italian, F1 of 0.484) and best language (Turkish, F1 of 0.628) differ by 30 percent in relative terms. For a moderation team, this gap is operationally important: per language FPR translates directly into per language false positive volume, which is a per market user experience cost. We recommend that any reported metric in a moderation context include both the overall number and the worst language number, and that thresholds be tuned to bound the worst case rather than to maximize the average.

Fourth, calibration data is cheap. Strategy D requires only a few thousand labeled examples per target language, distributed once at integration time; this is far cheaper than the labeled training data required to fine tune a multilingual encoder per language. For new languages added at deployment time (the test set scenario, with ru, fr, pt unseen during val tuning), Strategy B is the temporary fallback until the moderation team can label a small per language calibration set; switching to D after that will produce a measurable F1 lift without retraining the model.

5. Conclusion

We studied zero shot cross lingual toxic comment detection with English only training data, comparing four model families and four threshold strategies. Three findings are worth emphasizing. First, multilingual pretraining is the primary driver of cross lingual generalization: XLM R reaches 0.8363 ROC AUC and 0.7753 to 0.9259 per language AUC, while English only baselines stay at 0.52 to 0.66 per language AUC. Second, the canonical single threshold design (Strategy B, tuned on English dev) fails on every model in our comparison because the positive rate distribution shift from English dev (8.18 percent) to multilingual val (15.38 percent) destroys the threshold's recall guarantee. Third, per language calibration is the right fix, and a leak free version of it (Strategy D) recovers nearly all of the oracle gain (C minus D at most 0.011 F1 across all four models), making it both deployable and theoretically well founded.

For practitioners building multilingual moderation systems, the recommendation is concrete: use a multilingually pretrained encoder, hold out a stratified per language slice of validation data for threshold tuning, evaluate on the remaining slice, and treat the threshold as a first class engineering choice rather than a default. Our recommended operating point is XLM R combined with Strategy D, achieving F1 of 0.516 at FPR of 0.196 on val eval, with per language thresholds in the 0.017 to 0.028 range. All code and intermediate outputs needed to reproduce these results are available in our project repository (Group 5, 2025).

6. Limitations

6.1. Zero shot transfer constraints

Models are trained entirely on English Wikipedia talk pages and Civil Comments data. Toxicity expressions vary significantly across cultures; English derived patterns do not capture code switching, language specific slang, or culturally contextual insults. The validation set covers only three languages (es, it, tr), while the official test set introduces three additional unseen languages (ru, fr, pt). Threshold choices tuned on the validation languages may not transfer; Strategy D explicitly assumes per language tuning data is available, which is not the case for languages first appearing at test time.

6.2. Threshold calibration gap

Strategy B reveals that thresholds tuned on English dev (8.18 percent toxic) systematically fail on multilingual val (15.38 percent toxic), with actual recall far below the 0.7 target on every model. This is a structural limitation of any single threshold deployment, not a tuning bug. Strategy D's 50 to 50 val split also leaves only about 1,300 rows per language for threshold tuning, small enough that BiLSTM's per language tuner overshoots dramatically (recall 0.915 much greater than 0.7 with FPR 0.858), because the small per language tuning set fails to penalize the runaway false positive rate. This is a sample size limitation rather than a methodology one, but it is real. The original test set evaluation also uses a mixed thresholding setup: Spanish, Italian, and Turkish use Strategy D per-language thresholds, while French, Portuguese, and Russian use the Strategy B global fallback threshold because they were not present in the validation threshold tuning set. Therefore, the test result should be interpreted as a realistic deployment-style check, rather than a pure Strategy D evaluation for all six languages.

6.3. Domain and data shift

Training data from Wikipedia talk pages and Civil Comments may not match the platform contexts of the multilingual val and test data. The validation toxic rate (15.38 percent) is nearly 2 times the training rate (8.18 percent), which (combined with the language shift) may inflate apparent model performance on val relative to real world deployment, where the positive rate, content style, and platform conventions can all differ.

6.4. Model level constraints

Sequence length is fixed at 128 tokens for all neural models (BiLSTM, mBERT, XLM R); longer comments are truncated, potentially losing toxicity signals that appear later in the comment. mBERT and XLM R pretraining corpora are not uniformly distributed across languages: Italian is the weakest language for XLM R (AUC equals 0.7753) despite Italian being typologically close to Spanish, which likely reflects lower toxic domain vocabulary coverage in the Italian pretraining slice. The BiLSTM uses an English trained Keras tokenizer; out of vocabulary rate is high for non Latin script tokens, which is consistent with its near chance Turkish performance.

6.5. Evaluation reliability

ROC AUC under heavy class imbalance can be overly optimistic; we therefore also report PR AUC (XLM R equals 0.4341, mBERT equals 0.3242, LR equals 0.2459, BiLSTM equals 0.2375) as a more conservative measure. Per language estimates are based on about 2,500 rows per language under Strategies A, B, C and about 1,250 rows per language for the val_eval split used by Strategy D, so per language conclusions, particularly small differences between models within a single language, should be interpreted cautiously.

References

- Group 5. Zero shot cross lingual toxic comment detection: project source code and reproducibility materials, 2025. GitHub repository. https://github.com/qiyazhao062-a11y/BT5153_GroupProject/tree/Full_Version
- Jigsaw and Conversation AI. Jigsaw multilingual toxic comment classification, 2020. Kaggle competition. <https://www.kaggle.com/c/jigsaw-multilingual-toxic-comment-classification>
- Jigsaw and Conversation AI. Jigsaw unintended bias in toxicity classification, 2019. Kaggle competition. <https://www.kaggle.com/c/jigsaw-unintended-bias-in-toxicity-classification>